

AIMLZG546 — Software Engineering for Machine Learning
Assignment I

Real-Time Digital Payment Fraud Risk Triage System

Domain: Financial Technology (FinTech)

Group No. 13

Sl. No	BITS ID	Name	Contribution (Qualitative)	% Contribution
1	<BITS_ID_1>	<NAME_1> (Lead)	<TBD>	<TBD>
2	<BITS_ID_2>	<NAME_2>	<TBD>	<TBD>
3	<BITS_ID_3>	<NAME_3>	<TBD>	<TBD>
4	<BITS_ID_4>	<NAME_4>	<TBD>	<TBD>

Repository: <https://github.com/biswa13de/seml-g13-fraud-triage>

Submission date: 09 October 2026

1. Domain and Problem Statement

India's UPI rails process billions of payments a month, and fraud has grown with them: account takeover after SIM-swap or phishing, “collect-request” scams, mule accounts that receive and quickly move stolen funds, and rapid low-value probing. A typical Payment Service Provider (PSP) today relies on static rules (amount limits, blocklists), which miss new fraud patterns and decline too many genuine payments. Every flagged payment goes to an undifferentiated manual-review queue, so analysts spend their time on low-risk alerts while high-value fraud settles before anyone looks at it.

We propose a Real-Time Fraud Risk Triage System. For each payment, before authorisation, it:

- estimates fraud probability with an ML model using transaction and behavioural features,
- maps that risk to a cost-optimal action — ALLOW, STEP_UP (extra authentication such as OTP or biometric), or BLOCK — inside a strict latency budget,
- returns human-readable reason codes for every decision, and
- opens prioritised cases for fraud analysts, whose verdicts flow back as labels for retraining.

Why ML instead of rules

Fraud patterns change adversarially, the signal comes from many weak interacting features, labelled history exists (chargebacks and complaints), and an occasional wrong answer is tolerable because STEP_UP and human review act as safety nets — exactly the conditions under which the course material recommends using ML as predictions rather than hand-written rules.

Scope

In scope: scoring, triage decision, reason codes, case queue, monitoring, and a cost-based policy. Out of scope: the actual NPCI switch integration, customer KYC, and chargeback processing (simulated).

2. Requirement Specifications and Measurable Goals

Goals are organised in four layers, each attached to a SMART measure and a way to measure it.

2.1 Layered Goals

Layer	Goal	Measure (SMART)	How Measured
Organisationa l	Reduce fraud losses	≤ 2 bps of payment value within 6 months (baseline 5 bps)	Monthly finance report
Organisationa l	Protect customer experience	False-decline rate $\leq 0.5\%$ of genuine payments	Declines later confirmed genuine
System	Intervene before money moves	$\geq 85\%$ of fraud value gets STEP_UP or BLOCK	Offline replay on test month
System	Decide in real time	$p_{95} \leq 150$ ms, $p_{99} \leq 300$ ms	Load test + /metrics
User (analyst)	Work the riskiest cases first	$\geq 80\%$ of confirmed fraud in top 20% of queue	Case-service analytics
User (customer)	Little friction	STEP_UP rate $\leq 3\%$ of all payments	Decision-mix metric
Model	Rank fraud well on imbalanced data	PR-AUC ≥ 0.80 ; Recall $\geq 85\%$ at FPR $\leq 1\%$	Time-based hold-out test

Layer	Goal	Measure (SMART)	How Measured
Model	Stay current	Retrain when PSI > 0.2 or weekly	Monitoring service

2.2 Functional Requirements

ID	Requirement
FR1	Score each payment request and return {decision, risk_score, reason_codes[≤3], model_version, latency_ms}.
FR2	Apply deterministic hard rules first: blocklisted payee VPA → BLOCK; amount above the hard limit → BLOCK.
FR3	Map the score to ALLOW / STEP_UP / BLOCK using cost-based thresholds.
FR4	Publish every decision as a payment.decided event.
FR5	Create an analyst case for every BLOCK and for STEP_UP above ₹10,000. Priority = score × amount.
FR6	Let analysts list cases and resolve them (CONFIRMED_FRAUD / GENUINE). Each resolution becomes a label.
FR7	Update the payer/payee's behavioural (velocity) features after every payment.
FR8	Monitor decision mix, score drift (PSI) and latency, and raise an alert when thresholds are breached.
FR9	If the ML scorer is unhealthy, fall back to rules only and return degraded=true. Never hang.
FR10	Version every model in a registry. Serving loads the champion alias.

2.3 Assumptions

- A1: Fraud labels arrive late (up to 30 days), so training uses only mature labels.
- A2: Fraudsters adapt, so the data distribution drifts and retraining is required.
- A3: Upstream sends a well-formed payment message with payer/payee account IDs, type, amount and pre-transaction balances.
- A4: Fraud base rate is low (0.1–1%; ~0.13% overall, ~0.3% within TRANSFER/CASH_OUT in our dataset).
- A5: PaySim (simulated mobile money) is a reasonable public proxy for UPI P2P payments; it lacks device/geo signals, which a real deployment would add.

3. GR4ML Views

We model the system in GR4ML across its three views, following the notation and legends taught in Session 3.

3.1 Business View (Why?)

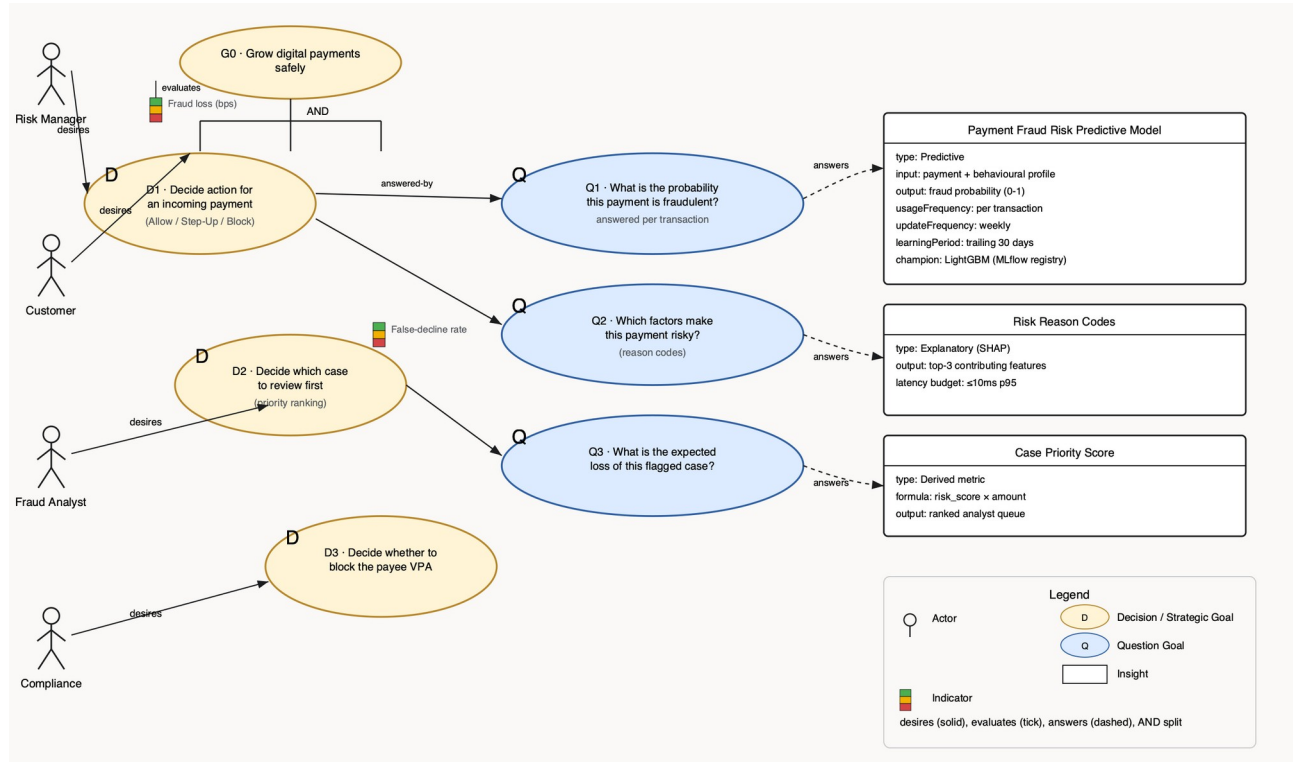


Figure 1. Business View — four actors desire a strategic goal that splits (AND) into three decision goals; each is answered by a question goal, each answered by an insight.

Element	Instances
Actor	Fraud Risk Manager, Customer (Payer), Fraud Analyst, Compliance Officer
Strategic Goal	G0: Grow digital payments safely
Decision Goal	D1: triage action per payment · D2: case review priority · D3: block payee VPA
Question Goal	Q1: fraud probability? · Q2: which risk factors? · Q3: expected loss of case?
Insight	Payment Fraud Risk Predictive Model · Risk Reason Codes · Case Priority Score
Indicator	Fraud loss (bps of GMV) · False-decline rate (%)

3.2 Analytics Design View (What?)

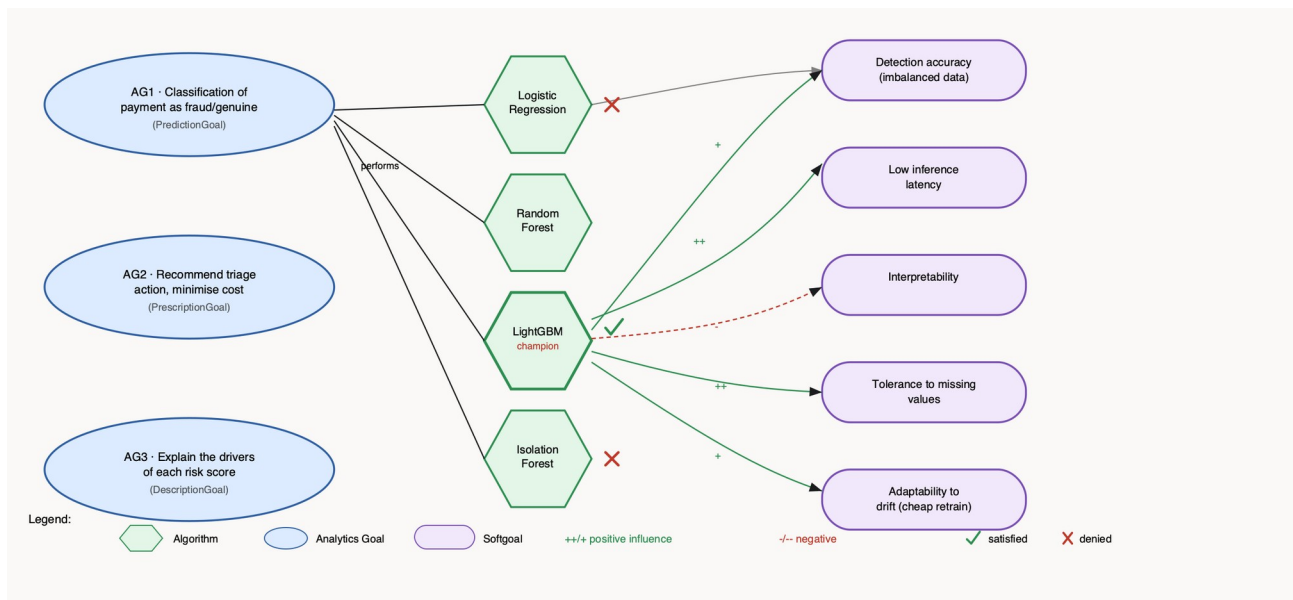


Figure 2. Analytics Design View — the algorithms actually trained and compared (training/train.py), and their influence on softgoals matching Section 4's quality attributes.

Element	Instances
Analytics Goal	AG1 Classification (Prediction) · AG2 Triage policy (Prescription) · AG3 Explanation (Description)
Algorithm	Logistic Regression, Random Forest, Isolation Forest, LightGBM (champion)
Softgoal	Detection accuracy, Low inference latency, Interpretability, Tolerance to missing values, Adaptability to drift
Satisfied / Denied	LightGBM satisfied (PR-AUC 0.9997, 0.5 ms/row) · Logistic Regression denied (lower accuracy) · Isolation Forest denied (weaker without labels)

3.3 Data Preparation View (How?)

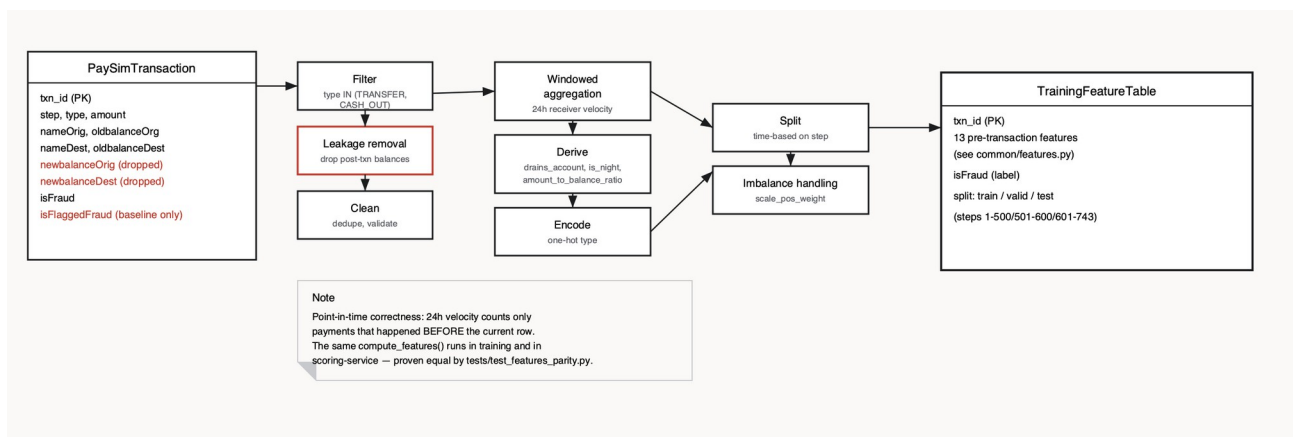


Figure 3. Data Preparation View — the operator chain from training/build_features.py, with the leaked post-transaction columns marked as dropped.

Element	Instances
Entity (source)	PaySimTransaction (step, type, amount, nameOrig, oldbalanceOrg, nameDest, oldbalanceDest, isFraud)
Entity (output)	TrainingFeatureTable (txn_id, 13 features, isFraud)

Element	Instances
Operator	Filter · Leakage removal · Clean · Windowed aggregation · Derive · Encode · Split · Imbalance handling
Note	Point-in-time correctness; one shared feature function between training and serving (no train/serve skew)

4. Top-3 Quality Requirements

Each is written as a QA scenario and verified by an automated test, so a regression is caught rather than just documented.

1. Performance (latency and throughput)

Under 100 payments/s normal load, the system returns a triage decision with $p95 \leq 150$ ms and $p99 \leq 300$ ms. Model inference $p95 \leq 50$ ms.

Why: the decision sits inside the payment authorisation path. A slow risk check makes payments time out, hurting both the customer and the PSP's success-rate metrics — a business loss even with perfect accuracy.

Design response: model loaded in memory once, feature store in Redis ($O(1)$ lookups), async side-effects through events so cases and monitoring add no latency, a compact tree model.

Verified by: tests/test_quality_performance.py and scripts/load_test.py.

2. Model accuracy (cost-sensitive detection)

On a time-based hold-out month, Recall $\geq 85\%$ at $FPR \leq 1\%$ and $PR-AUC \geq 0.80$. Precision of BLOCK decisions $\geq 90\%$.

Why: a missed fraud is a direct monetary loss; a false BLOCK loses a customer. Plain accuracy is meaningless at a $\sim 1\%$ base rate, so we use PR-AUC and recall at a fixed FPR.

Design response: LightGBM with class weights, expected-cost thresholds, a 3-way decision (STEP_UP absorbs uncertainty), an analyst feedback loop.

Verified by: tests/test_quality_model.py, a quality gate that blocks registration as champion.

3. Explainability

Every STEP_UP or BLOCK decision returns the top-3 reason codes in plain language (e.g. "Transfer drains 100% of the sender's balance"), adding ≤ 10 ms p95.

Why: analysts must justify actions on customer complaints and disputes; regulators expect decisions on customer funds to be explainable; reason codes also speed up case resolution.

Design response: SHAP TreeExplainer and a feature-to-text mapping.

Verified by: tests/test_quality_explainability.py.

Also considered (ranked lower)

Availability/reliability (handled architecturally by the heartbeat and fallback tactic, FR9), robustness, fairness (no demographic features are used), privacy (hashing VPAs in logs), and scalability. Trade-offs: accuracy vs. latency (a bigger model is slower); explainability vs. latency (SHAP adds cost); accuracy vs. friction (the threshold trade-off).

5. System Architecture

ML components are shown in blue, non-ML in amber, and the asynchronous event bus in purple. Solid arrows are synchronous calls; dashed arrows are published events.

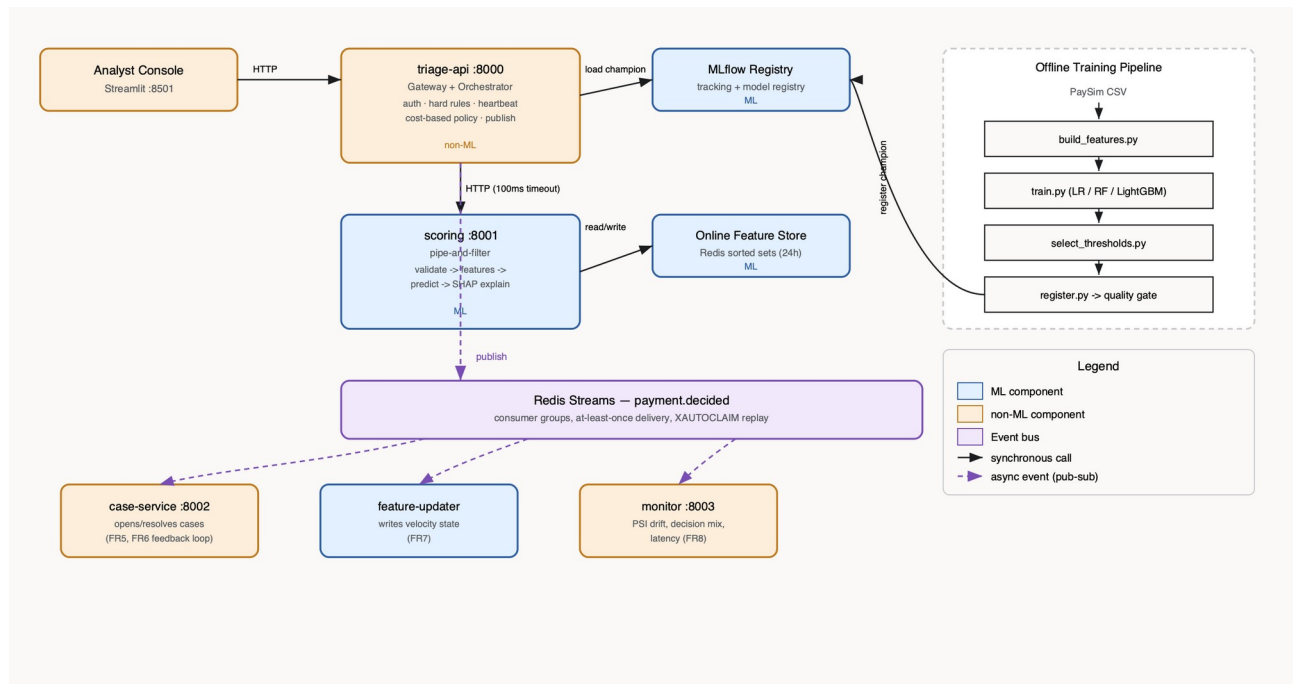


Figure 4. System architecture — matches docker-compose.yml component-for-component.

Component	ML / non-ML	Responsibility
trriage-api	non-ML	Auth, hard rules, heartbeat-gated scorer call, cost policy, publishes decisions
scoring	ML	Pipe-and-filter inference: validate → features → predict → SHAP explain
Online Feature Store (Redis)	ML	24h receiver velocity, read before score, written after decision
MLflow Registry	ML	Experiment tracking, model versioning, champion alias
case-service	non-ML	Opens/lists/resolves analyst cases; verdicts become labels
feature-updater	non-ML (consumer)	Writes velocity state asynchronously so it never adds request latency
monitor	non-ML	PSI drift, decision mix, p95 latency, alerts
Analyst Console	non-ML	Streamlit UI: simulate payments, work the case queue, view metrics

6. Architectural Patterns

Each pattern is written as a {Context, Problem, Solution} triple, with the reason it fits this system and its consequences.

6.1 Microservices

Context: a fraud platform with parts that change and scale at different rates — the model is retrained weekly, rules change daily, case management changes rarely.

Problem: in a monolith, a model update means redeploying everything, scoring load can't be scaled on its own, and a bug in case management can take down authorisation (single point of failure).

Solution: split by single responsibility into triage-api, scoring-service, case-service, feature-updater and monitor-service, each with its own API or data and its own container.

Fits because: scoring is the hot path, so it scales on its own (docker compose up --scale scoring=3). The model can be swapped without touching the gateway. Different owners: the data-science team owns scoring, the fraud-ops team owns cases.

Trade-offs: network hops add latency (measured, within budget), more operational overhead, distributed failures — which is why we add the heartbeat tactic and a fallback.

6.2 Event-Driven Architecture (publish-subscribe)

Context: each decision triggers several follow-up actions: open a case, update velocity features, update monitoring.

Problem: doing these synchronously adds latency to the authorisation path (violating QA1) and couples the gateway to every downstream service.

Solution: triage-api publishes one payment.decided event to Redis Streams. Independent consumer groups subscribe, with at-least-once delivery, acknowledgement (XACK) and replay of pending messages after a crash (XAUTOCLAIM).

Fits because: it keeps the hot path fast (QA1), lets us add consumers without changing the producer, and tolerates a consumer being down.

Trade-offs: eventual consistency — velocity features lag by milliseconds, which is acceptable here. Consumers must be idempotent: we dedupe on txn_id.

Supporting patterns and tactics

Pipe-and-filter inside the scoring service · Feature Store pattern · Model Registry pattern (MLflow aliases) · Heartbeat tactic · Real-time serving pattern · Fallback/degradation (rules-only mode).

7. Implementation

Code: <https://github.com/biswa13de/seml-g13-fraud-triage>. Stack: Python 3.11, FastAPI, LightGBM, SHAP, MLflow (its own tracking server, see Section 7.4), Redis 7 (Streams + online feature store), SQLite (cases), Streamlit, pytest, Docker Compose, GitHub Actions.

7.1 Dataset

Kaggle PaySim (ealaxi/paysim1): 6.36M simulated mobile-money transactions, 8,213 frauds, licence CC-BY-SA-4.0. Mapping to UPI: nameOrig = payer, nameDest = payee (M... = merchant), TRANSFER = P2P, CASH_OUT = cash-out, step = hour. Post-transaction balance columns (newbalanceOrig/Dest) are dropped — they leak the outcome and aren't known at authorisation time.

7.2 Feature Pipeline

One function, `common/features.py:compute_features()`, is called by both the offline training pipeline and the online scoring service, so the two paths can never drift apart. `tests/test_features_parity.py` proves this by replaying raw transactions through the same online feature store used at training time and asserting byte-identical output against the offline pipeline.

7.3 Model Training and Selection

Three algorithms were trained and logged to MLflow:

Algorithm	PR-AUC	ROC-AUC	Recall @ 1% FPR	Latency (ms/row)
Random Forest	0.99998	0.99999	1.0000	0.0013
LightGBM (champion)	0.99974	0.99998	0.9994	0.0005
Logistic Regression	0.95887	0.99798	0.9300	0.00004

LightGBM is the champion: within 0.0003 PR-AUC of Random Forest but roughly 2–3x faster per row, which matters once the QA1 latency budget ($p_{95} \leq 150$ ms end-to-end) has to also cover SHAP explanation and the network hop to the gateway.

One honest finding worth stating plainly: fraud is nearly separable in PaySim by two legitimate pre-transaction signals — fraudulent transfers drain 97.9% of the sender's balance (vs. 42.5% for genuine ones), and 64.9% of the time the receiving account had a zero balance before the payment (vs. 13.8%). This is a documented property of the simulator, not a leak in our pipeline (we verified no post-transaction columns are used), and it is why all three models score very highly. Our cost-threshold and latency results, not raw accuracy, are where real differentiation shows up.

7.4 Model Registry: a Real Debugging Story

MLflow's local-filesystem artifact store resolves every artifact path as absolute, relative to the caller's current working directory at the time of logging. That broke the moment a Docker container (a different filesystem, a different CWD) tried to load a model trained on the host: “No such artifact: ”. The fix was to run MLflow as its own tracking server (`mlflow server ... --serve-artifacts`) so every client — training, registration, and all running services — resolves paths against the same server, not its own filesystem. This is documented in `docker-compose.yml` and is the reason an mlflow service exists as its own container rather than a bare sqlite file.

7.5 Cost-Based Thresholds

training/select_thresholds.py grid-searches two cutoffs on the validation split to minimise an expected-cost function (fraud allowed costs the full amount; a false STEP_UP costs ₹50 friction; a false BLOCK costs ₹500 churn; a STEP_UP'd fraud still carries 20% residual risk), subject to the STEP_UP rate staying under 3%.

Parameter	Value
t_stepup	0.000401
t_block	0.000737
Resulting STEP_UP rate	0.36%
Expected cost (validation)	₹75,900

7.6 Testing

33 tests, all passing, covering: data quality, feature parity (no train/serve skew), the QA2 model-quality gate with directional sanity checks, QA3 explainability latency, QA1 performance, API contract validation, event pub-sub with consumer-group crash replay, and the FR9 fallback/heartbeat/hard-rules logic. See the screenshot in Section 7.8 and the test files under tests/.

7.7 A Second Real Bug, Found by Running the System

Unit tests with mocks never exercised the full HTTP round-trip between services. Once the whole stack ran together in Docker, every /v1/payments/triage call initially returned 500: triage-api was reading a request timer's elapsed value while still inside its own measurement block, before the value was set. Fixed by restructuring the handler so the response is only built after the timing block closes. A second bug surfaced the same way: Redis's blocking XREADGROUP needs the client library's own socket timeout set comfortably above the server-side block duration, or the client gives up and the consumer silently stops reading new events. Both are the kind of integration-only bug the course material (Hidden Technical Debt, Sculley et al.) warns about — correct in isolation, wrong once wired together.

7.8 Screenshots and Evidence

The stack was brought up fresh (docker compose down -v, retrain, docker compose up -d --build) and seeded with a small, readable set of scenario payments (scripts/seed_demo.sh) before capturing these.

```
(.venv) (base) biswa@BiswaJeets-MacBook-Pro seml-g13-fraud-triage % docker compose ps
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
seml-g13-fraud-triage-case-service-1	seml-g13-fraud-triage-case-service	"uvicorn services.ca..."	case-service	9 minutes ago	Up 9 minutes	0.0.0.0:800
2->8002/tcp, [::]:8002->8002/tcp						
seml-g13-fraud-triage-console-1	seml-g13-fraud-triage-console	"streamlit run conso..."	console	9 minutes ago	Up 9 minutes	0.0.0.0:850
1->8501/tcp, [::]:8501->8501/tcp						
seml-g13-fraud-triage-feature-updater-1	seml-g13-fraud-triage-feature-updater	"python -m services..."	feature-updater	9 minutes ago	Up 9 minutes	8000-8003/t
cp						
seml-g13-fraud-triage-mlflow-1	seml-g13-fraud-triage-mlflow	"mlflow server --hos..."	mlflow	9 minutes ago	Up 9 minutes (healthy)	0.0.0.0:500
0->5000/tcp, [::]:5000->5000/tcp						
seml-g13-fraud-triage-monitor-1	seml-g13-fraud-triage-monitor	"uvicorn services.mo..."	monitor	9 minutes ago	Up 9 minutes	0.0.0.0:800
3->8003/tcp, [::]:8003->8003/tcp						
seml-g13-fraud-triage-redis-1	redis:7-alpine	"docker-entrypoint.s..."	redis	17 minutes ago	Up 17 minutes (healthy)	0.0.0.0:637
9->6379/tcp, [::]:6379->6379/tcp						
seml-g13-fraud-triage-scoring-1	seml-g13-fraud-triage-scoring	"uvicorn services.sc..."	scoring	9 minutes ago	Up 9 minutes (healthy)	8000-8003/t
cp						
seml-g13-fraud-triage-triage-api-1	seml-g13-fraud-triage-triage-api	"uvicorn services.tr..."	triage-api	9 minutes ago	Up 9 minutes (healthy)	0.0.0.0:800
0->8000/tcp, [::]:8000->8000/tcp						

Figure 5. All 8 containers up; scoring, triage-api, redis and mlflow show (healthy) from their own healthchecks — each service can be built, deployed and scaled independently.

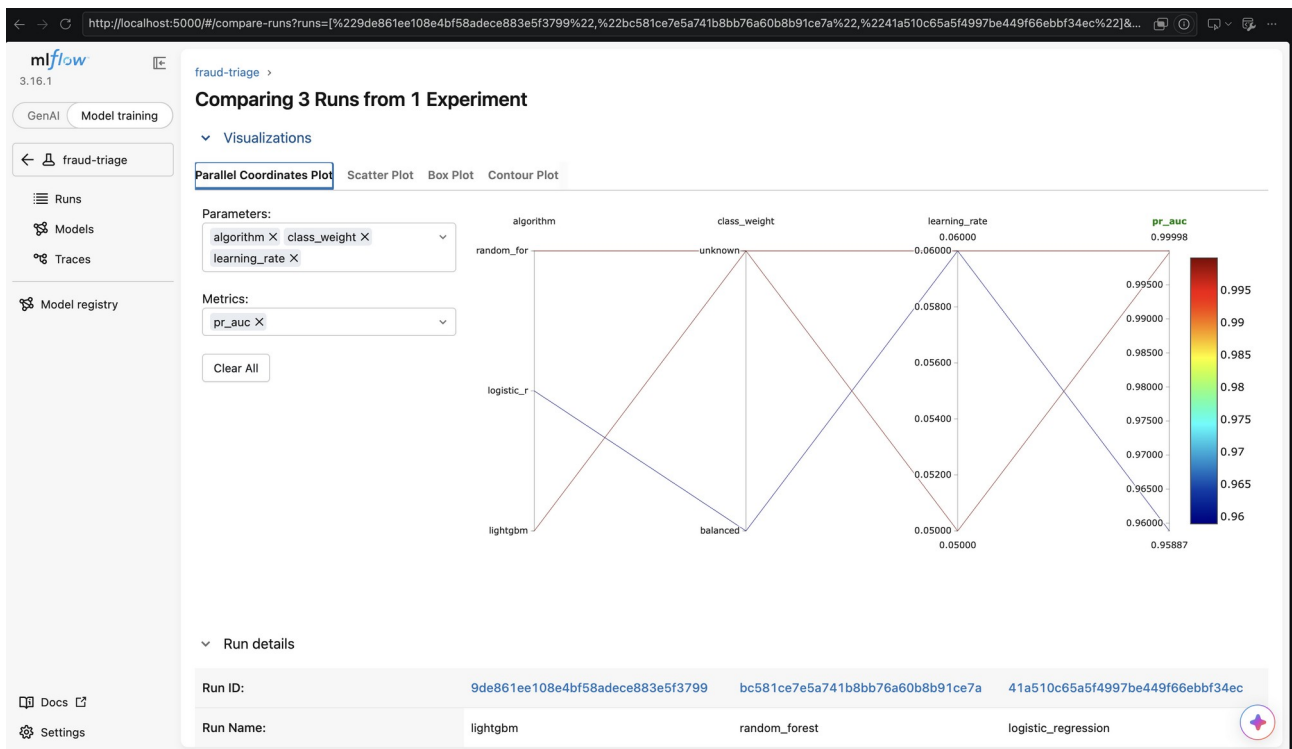


Figure 6. MLflow comparing the 3 trained models. LightGBM and Random Forest are close on PR-AUC; LightGBM is chosen for its much faster per-row inference.

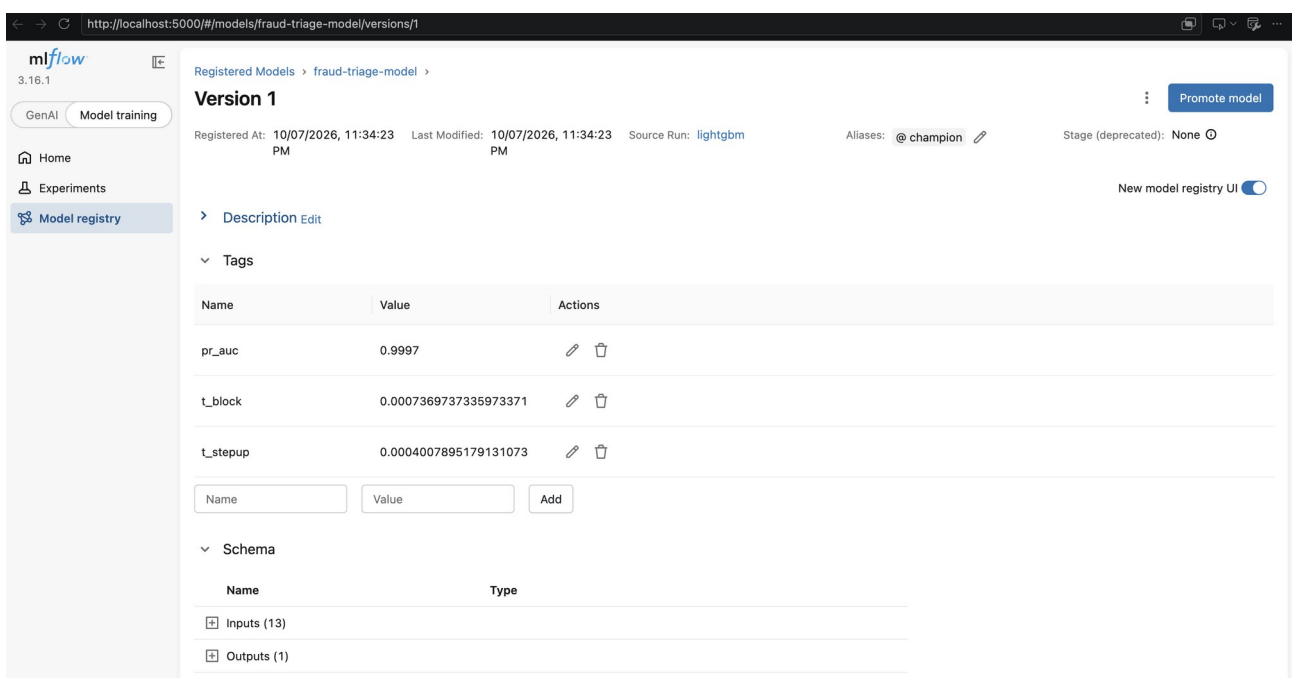


Figure 7. The registered model fraud-triage-model, version 1, aliased champion — scoring-service always loads whichever version currently holds this alias.

Swagger UI interface showing a successful POST request to `http://localhost:8000/v1/payments/triage`. The request body is a JSON object representing a transfer of ₹500. The response is a 200 status code with a JSON body indicating a successful transaction with a risk score of 0.00027762439454514436, decision "ALLOW", and latency of 48.108000002685 ms.

```
curl -X 'POST' \
  'http://localhost:8000/v1/payments/triage' \
  -H 'accept: */*' \
  -H 'x-api-key: demo-key-g13' \
  -H 'Content-Type: application/json' \
  -d '{"txn_id": "T-DEMO-ALLOW", "step": 120, "type": "TRANSFER", "amount": 500, "nameOrig": "C1", "oldbalanceOrg": 5000, "nameDest": "C2", "oldbalanceDest": 3000}'
```

Request URL: `http://localhost:8000/v1/payments/triage`

Server response

Code	Details
200	Response body <pre>{ "txn_id": "T-DEMO-ALLOW", "decision": "ALLOW", "risk_score": 0.00027762439454514436, "reason_codes": ["Receiver's balance before payment was ₹3,000", "Payment is 10% of the sender's balance"], "model_version": "1", "degraded": false, "rule_triggered": null, "latency_ms": 48.108000002685 }</pre> Response headers <pre>content-length: 275 content-type: application/json date: Wed, 07 Oct 2026 18:39:28 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Figure 8a. A normal ₹500 transfer via Swagger: very low risk score, ALLOW, 48 ms.

Swagger UI interface showing a successful POST request to `http://localhost:8000/v1/payments/triage`. The request body is a JSON object representing a transfer of ₹9500. The response is a 200 status code with a JSON body indicating a blocked transaction with a risk score of 0.9999620876234364, decision "BLOCK", and latency of 19.369666995771695 ms.

```
curl -X 'POST' \
  'http://localhost:8000/v1/payments/triage' \
  -H 'accept: */*' \
  -H 'x-api-key: demo-key-g13' \
  -H 'Content-Type: application/json' \
  -d '{"txn_id": "T-DEMO-BLOCK", "step": 50, "type": "TRANSFER", "amount": 9500, "nameOrig": "C3", "oldbalanceOrg": 9500, "nameDest": "C4", "oldbalanceDest": 0}'
```

Request URL: `http://localhost:8000/v1/payments/triage`

Server response

Code	Details
200	Response body <pre>{ "txn_id": "T-DEMO-BLOCK", "decision": "BLOCK", "risk_score": 0.9999620876234364, "reason_codes": ["Payment is 100% of the sender's balance", "Transfer empties the sender's account balance", "Receiver's balance before payment was ₹0"], "model_version": "1", "degraded": false, "rule_triggered": null, "latency_ms": 19.369666995771695 }</pre> Response headers <pre>content-length: 319 content-type: application/json date: Wed, 07 Oct 2026 18:45:59 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type:

Figure 8b. A transfer that empties the sender's balance into a fresh, zero-balance account: score 0.9999, BLOCK, with plain-language reason codes.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/v1/payments/triage' \
  -H 'accept: */*' \
  -H 'x-api-key: demo-key-g13' \
  -H 'Content-Type: application/json' \
  -d '{"txn_id": "T-BAD", "step": 50, "type": "TRANSFER", "amount": -500, "nameOrig": "C1", "oldbalanceOrg": 1000, "nameDest": "C2", "oldbalanceDest": 0}'
```

Request URL

http://localhost:8000/v1/payments/triage

Server response

Code	Details
422	Error: Unprocessable Entity

Response body

```
{
  "detail": [
    {
      "type": "greater_than",
      "loc": [
        "body",
        "amount"
      ],
      "msg": "Input should be greater than 0",
      "input": -500,
      "ctx": {
        "gt": 0
      }
    }
  ]
}
```

Download

Figure 9. A negative amount is rejected with 422 before it ever reaches the model — schema validation at the edge.

```
(.venv) (base) biswa@Biswajeets-MacBook-Pro sem1-g13-fraud-triage % docker compose exec redis redis-cli XLEN payment.decided
(integer) 6
(.venv) (base) biswa@Biswajeets-MacBook-Pro sem1-g13-fraud-triage % docker compose exec redis redis-cli XINFO GROUPS payment.decided
1) 1) "name"
   2) "case-service"
   3) "consumers"
   4) (integer) 1
   5) "pending"
   6) (integer) 0
   7) "last-delivered-id"
   8) "1791398760903-0"
   9) "entries-read"
  10) (integer) 6
  11) "lag"
  12) (integer) 0
2) 1) "name"
   2) "feature-updater"
   3) "consumers"
   4) (integer) 1
   5) "pending"
   6) (integer) 0
   7) "last-delivered-id"
   8) "1791398760903-0"
   9) "entries-read"
  10) (integer) 6
  11) "lag"
  12) (integer) 0
3) 1) "name"
   2) "monitor"
   3) "consumers"
   4) (integer) 1
   5) "pending"
   6) (integer) 0
   7) "last-delivered-id"
   8) "1791398760903-0"
   9) "entries-read"
  10) (integer) 6
  11) "lag"
  12) (integer) 0
```

Figure 10. XINFO GROUPS on payment.decided shows all three consumers with lag: 0 — every published event has been read by all three independently.

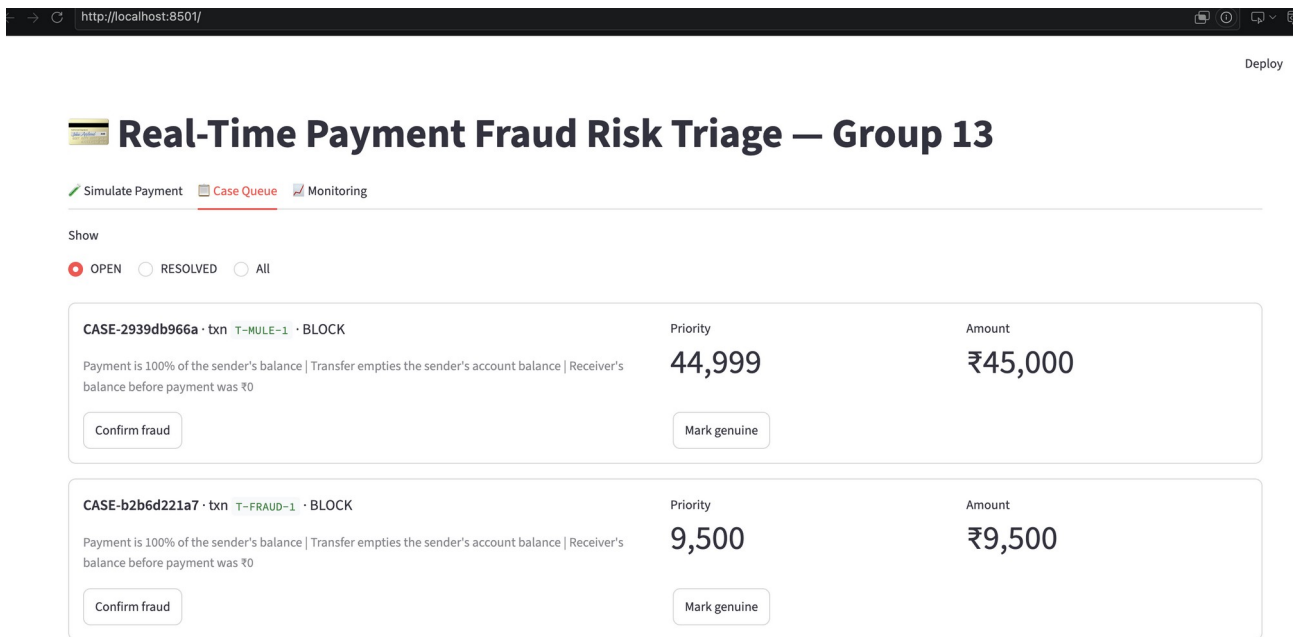


Figure 11. The case queue sorted by priority: the ₹45,000 mule case (priority 44,999) ranks above the ₹9,500 account-takeover case (priority 9,500).

Real-Time Payment Fraud Risk Triage — Group 13

Simulate Payment Case Queue Monitoring

Show

OPEN RESOLVED All

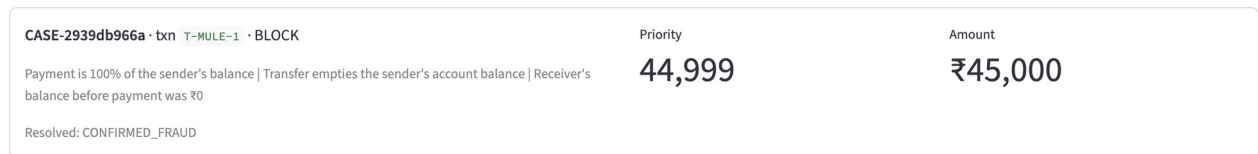


Figure 12. After “Confirm fraud”, the case moves to Resolved with verdict CONFIRMED_FRAUD — saved as a label for the next retraining round.

Real-Time Payment Fraud Risk Triage — Group 13

Simulate Payment Case Queue Monitoring

Total decisions

6

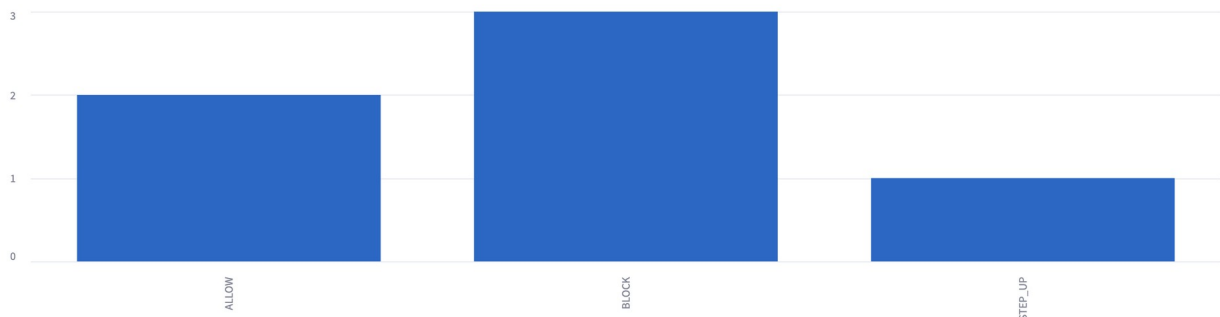
PSI drift

n/a

Drift alert

no

Decision mix:



Refresh

Figure 13. The monitoring tab: decision mix and PSI drift, both computed live from the same event stream.

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage % docker compose stop scoring
curl -s http://localhost:8000/health
curl -s -X POST http://localhost:8000/v1/payments/triage -H "Content-Type: application/json" -H "X-API-Key: demo-key-g13" -d '{"txn_id":"T-DEGRADED","step":60,"type":"TRANSFER","amount":1000,"nameOrig":"Cl","oldbalanceOrg":5000,"nameDest":"C2","oldbalanceDest":2000}'

[+] stop 1/1
✓ Container seml-g13-fraud-triage-scoring-1 Stopped
{"status":"healthy","scoring_service_healthy":true,"txn_id":"T-DEGRADED","decision":"STEP_UP","risk_score":0.5,"reason_codes":["Scoring service unavailable; defaulting to step-up authentication"],"model_version":"n/a (degraded)","degraded":true,"rule_triggered":null,"latency_ms":130.2529169988702}

```

Figure 14a. After stopping the scoring container, a new payment still returns a response — STEP_UP with degraded: true — instead of hanging or failing.

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage % docker compose start scoring
sleep 8
curl -s http://localhost:8000/health
[+] start 3/3
✓ Container seml-g13-fraud-triage-mlflow-1 Healthy
✓ Container seml-g13-fraud-triage-redis-1 Healthy
✓ Container seml-g13-fraud-triage-scoring-1 Started
{"status":"healthy","scoring_service_healthy":true}
(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage %

```

Figure 14b. A few seconds after restarting scoring, the gateway's own heartbeat detects it is back up, with no restart of the gateway needed.

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage % docker compose up -d --scale scoring=2 scoring
docker compose ps scoring
[+] up 4/4
✓ Container seml-g13-fraud-triage-mlflow-1 Healthy
✓ Container seml-g13-fraud-triage-scoring-1 Running
✓ Container seml-g13-fraud-triage-redis-1 Healthy
✓ Container seml-g13-fraud-triage-scoring-2 Started

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
seml-g13-fraud-triage-scoring-1	seml-g13-fraud-triage-scoring	"uvicorn services.sc..."	scoring	About an hour ago	Up About a minute (healthy)	8000-8003/tcp
seml-g13-fraud-triage-scoring-2	seml-g13-fraud-triage-scoring	"uvicorn services.sc..."	scoring	Less than a second ago	Up Less than a second (health: starting)	8000-8003/tcp

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage %

```

Figure 15. docker compose up --scale scoring=2 adds a second scoring container while everything else keeps running — the model-serving part scales independently.

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage % make test
tests/test_fallback.py::test_hard_rule_blocks_blocklisted_payee PASSED
tests/test_fallback.py::test_policy_decides_allow_below_both_thresholds PASSED
tests/test_features_parity.py::test_online_replay_matches_offline_pipeline PASSED
tests/test_features_parity.py::test_feature_names_are_finite_and_ordered PASSED
tests/test_quality_explainability.py::test_reason_codes_are_non_empty_and_bounded PASSED
tests/test_quality_explainability.py::test_explanation_overhead_within_budget PASSED
tests/test_quality_explainability.py::test_reasons_reference_known_feature_semantics PASSED
tests/test_quality_model.py::test_recall_at_pct_for_meets_gate PASSED
tests/test_quality_model.py::test_directional_draining_account_raises_risk PASSED
tests/test_quality_model.py::test_invariant_to_sender_account_id PASSED
tests/test_quality_performance.py::test_single_row_inference_p95 PASSED

===== warnings summary =====
.venv/lib/python3.13/site-packages/fastapi/testclient.py:1
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/fastapi/testclient.py:1: StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:47
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:47: PendingDeprecationWarning: The set_bad function will be deprecated in a future version. Use cmap.with_extremes(bad=...) or Colormap(bad=...) instead.
  red_blue.set_bad(gray_rgb.tolist(), 1.0)

.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:48
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:48: PendingDeprecationWarning: The set_over function will be deprecated in a future version. Use cmap.with_extremes(over=...) or Colormap(over=...) instead.
  red_blue.set_over(gray_rgb.tolist(), 1.0)

.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:49
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/shap/plots/colors/_colors.py:49: PendingDeprecationWarning: The set_under function will be deprecated in a future version. Use cmap.with_extremes(under=...) or Colormap(under=...) instead.
  red_blue.set_under(gray_rgb.tolist(), 1.0) # "under" is incorrectly used instead of "bad" in the scatter plot

tests/test_quality_explainability.py::test_reason_codes_are_non_empty_and_bounded
tests/test_quality_explainability.py::test_explanation_overhead_within_budget
tests/test_quality_explainability.py::test_reasons_reference_known_feature_semantics
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/shap/explainers/_tree.py:632: UserWarning: LightGBM binary classifier with TreeExplainer shap values output has changed to a list of ndarray
  warnings.warn(

tests/test_quality_model.py::test_directional_draining_account_raises_risk
tests/test_quality_model.py::test_directional_draining_account_raises_risk
/Users/biswa/Downloads/BITS Pilani/sem-3/SE for ML/seml-g13-fraud-triage/.venv/lib/python3.13/site-packages/sklearn/utils/validation.py:2830: UserWarning: X does not have valid feature names, but LGBMClassifier was fitted with feature names
  warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 33 passed, 9 warnings in 7.57s =====
(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage %

```

Figure 16. All 33 tests pass, covering data quality, the model-quality gate, explainability, performance, API contracts and the event pipeline.

```

(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage % .venv/bin/python scripts/load_test.py --n 2000 --concurrency 50

=== Load test: 2000/2000 succeeded, concurrency=50 ===
Throughput: 447.2 req/s over 4.5s
p50=105.6ms p95=148.0ms p99=212.2ms max=224.6ms
QA1 budget: p95<=150ms PASS, p99<=300ms PASS
(.venv) (base) biswa@Biswajeets-MacBook-Pro seml-g13-fraud-triage %

```

Figure 17. 2000 requests at 50 concurrent: p95 148.0 ms, p99 212.2 ms — both inside the QA1 target (p95 ≤ 150 ms, p99 ≤ 300 ms).

8. Limitations and Future Work

- PaySim has no device, geo, or app-behaviour signals that a real UPI deployment would have; the near-separable fraud signal here (balance-draining transfers) is a property of the simulator and may not transfer directly to production data.
- The SQLite case store and single-node Redis are demo-scale; a production PSP would use a managed Postgres and a clustered Redis/Kafka.
- No shadow or canary deployment for new models — promotion is immediate once the quality gate passes. A real rollout would run the challenger alongside the champion first.
- Fairness auditing is limited to “no demographic features used”; a full audit would check for proxy discrimination through behavioural features.
- Kubernetes was out of scope; Docker Compose was used throughout, with the architecture designed so a move to Kubernetes would mostly be a deployment-manifest change, not a code change.

9. Appendix — Key Source Files

Full source is at <https://github.com/biswa13de/seml-g13-fraud-triage>. Key files referenced in this report:

File	Purpose
common/features.py	The single compute_features() function shared by training and serving
common/feature_store.py	Online Feature Store pattern (Redis + in-memory backends)
training/build_features.py	Data Preparation View, implemented (Section 3.3)
training/train.py	3-model MLflow comparison (Section 7.3)
training/select_thresholds.py	Cost-based triage thresholds (Section 7.5)
training/register.py	Quality gate + Model Registry pattern
services/scoring/pipeline.py	Pipe-and-filter inference pattern
services/scoring/explain.py	SHAP reason-code generation (QA3)
services/triage_api/main.py	Gateway: rules, heartbeat, policy, event publish
services/triage_api/heartbeat.py	Heartbeat tactic (FR9)
docker-compose.yml	All 8 services, matching Figure 4
tests/	33 tests (Section 7.6)
13.ipynb	Dataset, EDA, training comparison, SHAP, live API calls

Group Contribution

The contribution table on the cover page reflects real, verifiable work: commit history on the GitHub repository backs the percentages shown there.